

Identifying Cryptomining Attempts on University Infra: A Machine Learning Approach

Megh Rathod
mr7375@nyu.edu
New York University
New York, United States

Abstract

Illicit cryptomining on shared high-performance computing (HPC) clusters poses a growing challenge for universities, leading to wasted computational resources, inflated energy costs, and reduced system availability for legitimate research. This paper presents a hybrid detection framework that integrates host-level telemetry and network traffic analysis to identify unauthorized mining workloads across the NYU campus network. Our approach combines rule-based heuristics with machine-learning classifiers trained on large-scale datasets, including MineShark and locally collected HPC telemetry. By fusing sustained utilization metrics, hardware-performance-counter patterns, and encrypted flow features, the system distinguishes mining behavior from legitimate compute-intensive tasks such as machine-learning training. We outline a scalable architecture based on Prometheus and Nagios for real-time monitoring, a probabilistic decision layer for detection, and an adaptive feedback loop for continual retraining. Preliminary evaluation design targets over 95% detection accuracy with less than 1% false-positive rate, demonstrating the feasibility of deploying AI-assisted cryptomining detection in academic computing environments.

Keywords

Cryptomining detection, HPC monitoring, network traffic analysis, hardware performance counters, hybrid AI-rule systems, university cybersecurity, anomaly detection.

1 Introduction

Cryptocurrencies have emerged over the past decade as a new paradigm of digital finance, because of its decentralized, peer-to-peer cryptographic nature compared to the central banks. At the heart of many cryptocurrencies (notably Bitcoin) lies the process of *mining*: using computational work to validate transactions, add new blocks to the blockchain, and earn rewards in the form of newly minted coins and transaction fees. Mining thus serves a dual role: securing the network and issuing new currency units. On one hand, mining is essential for the operation of proof-of-work (PoW) blockchains, on the other, the underlying economics make large-scale mining a highly lucrative business.

From a financial perspective, the scale of cryptocurrency mining is immense. Between January 1 and July 23 2024, Bitcoin miners alone generated approximately **12,970 BTC** (roughly **\$863 million USD**) in transaction fees alone [1]. Global mining revenues, when including block rewards, are estimated in the tens of billions of dollars annually. Industry trackers report that Bitcoin mining alone generated over **\$13 billion** in recent years, with daily industry revenues averaging around **\$50 million** or more [2]. The

operational costs of mining, however, vary significantly by geography: mining one Bitcoin can cost as little as **\$1,320** in Iran due to subsidized electricity, but over **\$306,000** in Italy [3]. Electricity remains the dominant cost component, with some analyses estimating that a single Bitcoin transaction consumes as much energy as an average U.S. household uses in over a month [4]. These numbers underscore both the economic scale and the environmental footprint of mining operations.

Universities typically provide robust computing infrastructure to students, faculty, and researchers to facilitate computationally intensive workloads in science, engineering, and data analysis. Many campuses host high-performance computing (HPC) clusters, GPU-equipped research servers, and gigabit-scale campus networks. Access to such resources is usually authenticated via enterprise-grade identity systems such as *802.1X* Wi-Fi and centralized *NetID*-based logins. While these facilities are indispensable for academic research, they also present an attractive target for abuse. A technically skilled student or researcher with legitimate access could exploit underutilized compute cycles or idle GPU nodes to conduct unauthorized cryptomining.

Cryptomining’s profitability, combined with virtually zero cost of power and computing resources for students in institutional environments, creates a strong incentive for misuse. Mining processes are computationally intensive and typically run continuously, often 24/7, consuming CPU/GPU resources and generating long-lived outbound network connections to mining pools. Such activity can degrade service quality for legitimate users, inflate electricity and cooling costs, and, in extreme cases, damage hardware due to sustained thermal loads. Moreover, mining traffic is often encrypted or tunneled to evade detection, making conventional signature-based security measures ineffective.

Given these challenges, there is a growing need for intelligent, automated systems capable of detecting unauthorized mining on shared infrastructure. These systems must operate efficiently at scale, tolerate encryption and traffic obfuscation, and crucially, attribute suspicious activity to specific users and devices. In this work, we propose a hybrid approach that combines rule-based heuristics with machine learning to detect cryptomining on university infrastructure. Our framework integrates host-level utilization metrics, network flow data, and authenticated user sessions, allowing for cross-layer correlation between computational behavior and network activity. By leveraging existing enterprise authentication systems (such as *802.1X* Wi-Fi and centralized *NetID* credentials), the proposed approach enables precise attribution and actionable alerts, empowering administrators to identify and mitigate unauthorized mining while minimizing false positives.

2 Related Work

Cryptomining detection research spans host-based and network-based techniques, reflecting complementary views on the problem. Host-based approaches monitor system-level indicators such as CPU/GPU utilization patterns, instruction sequences, hardware performance counters, or system call traces to identify mining processes directly on the endpoint. These methods offer high precision and can distinguish between legitimate computational workloads and illicit mining, but require agent deployment and may miss mining activity when processes are obfuscated or hardware monitoring is unavailable. In contrast, network-based approaches analyze traffic patterns, flow characteristics, and protocol signatures to infer mining activity from network metadata. These methods scale more easily across enterprise or campus environments, work on encrypted traffic, and require no endpoint access, but face challenges in attributing activity to specific users or devices and may produce false positives from benign long-lived connections. The following sections survey prominent contributions in both categories and identify gaps that motivate our hybrid, attribution-aware approach.

2.1 Host-Based Detection

(author?) [5] proposed one of the earliest empirical studies on dynamic cryptomining detection. Using sandboxed dynamic analysis of Windows executables, their system identified recurring hashing loops and CPU instruction patterns unique to mining operations. The model achieved close to 100% detection accuracy with negligible false positives on benign workloads such as browsers and office applications, highlighting the viability of instruction-level profiling.

(author?) [6] extended this direction by developing a deep learning-based hybrid static and dynamic detection model. They extracted opcode frequency vectors from binaries (static) and system call traces during execution (dynamic), feeding both into an LSTM-based classifier. The hybrid model improved detection accuracy from 95% (static only) to 99% (combined), demonstrating that miners exhibit distinguishable behavioral sequences when running.

To capture hardware-level signals, (author?) [7] utilized hardware performance counters (HPCs) to profile Proof-of-Work computations. By monitoring cache misses, instruction throughput, and memory-access entropy, they distinguished miners from benign CPU/GPU workloads even when processes were obfuscated. Their system, evaluated on seven major mining algorithms (including CryptoNight, Ethash, and Scrypt), achieved over 99% detection accuracy across diverse hardware setups using Random Forest classifiers.

Cloud and containerized environments introduce distinct behavioral patterns. (author?) [8] built an explainable ML framework using system call traces from Docker and Kubernetes pods. They constructed n-gram embeddings of syscall sequences and trained XGBoost and decision tree models. The system achieved 97% precision and recall across real cloud workloads, while explainability tools (LIME and SHAP) provided interpretable insights into anomalous pod behaviors indicative of mining. This approach demonstrated scalability in multi-tenant environments.

2.2 Network-Based Detection

At the network level, mining activities can be inferred from their distinct Stratum protocol communication and persistent, low-throughput flows. (author?) [9] developed XMR-Ray, which analyzes NetFlow metadata to detect illicit Monero mining traffic. Instead of deep packet inspection, it models temporal dependencies in flow sequences and Stratum request-response intervals. Their one-class classifier achieved 98.9% true positive rate and only 0.05% false positives, even under TLS-encrypted traffic. This confirms that Stratum timing features leak identifiable patterns.

(author?) [10] presented Crypto-Aegis, a metadata-only ML framework that uses flow-level features, packet sizes, inter-arrival times, and session durations, to classify cryptomining versus normal traffic. Evaluated across Bitcoin, Monero, and Bytecoin datasets, Crypto-Aegis achieved an F1 score of 0.96 and ROC-AUC of 0.99, proving effective even when traffic is encrypted or tunneled.

(author?) [11] broadened these findings through a feature-rich statistical model with 48 derived traffic attributes, including long-range dependence (Hurst exponent) and burstiness metrics. Using Random Forest and LightGBM models, they reported 99.3% precision and 98.8% recall on mixed mining and non-mining traffic datasets. Their results suggest that temporal self-similarity of mining traffic is a robust detection feature.

(author?) [12] proposed a hybrid network analysis that couples passive flow monitoring with active Stratum handshake validation. Their method first flags flows that match mining-like temporal features, then probes endpoints to confirm Stratum responses. This reduced false positives by over 40% compared to passive-only detectors, at the cost of increased probing overhead.

(author?) [13] adapted this hybrid logic to an encrypted traffic scenario in campus networks. Their two-stage system first employs an ML-based time-series classifier (XGBoost) trained on 231 flow features to detect suspicious encrypted connections, followed by active probing of suspected endpoints. The approach reached 99.4% recall and 99.3% accuracy on large-scale campus data while reducing false alarms by 35%.

2.3 Comparative Analysis and Open Gaps

Across these studies, host-based detectors provide fine-grained process-level visibility (Carlin, Darabian, Gangwal, Karn) but suffer from deployment and scaling challenges in shared clusters. Network-based detectors (Russo, Caprolu, Dong, Vesely, Shi) offer broader coverage and encryption tolerance, though they lack attribution capability and can misclassify long-running benign traffic.

A recurring research gap is the lack of cross-layer correlation, no existing system fuses host telemetry with network evidence to produce user-level attribution. Our work builds upon these foundations by proposing a hybrid rule-based and ML system that combines sustained CPU/GPU utilization, network flow metadata, and authenticated user identity (via 802.1X and NetID mapping). This fusion improves interpretability, reduces false positives, and enables actionable identification of unauthorized cryptomining on shared university infrastructure.

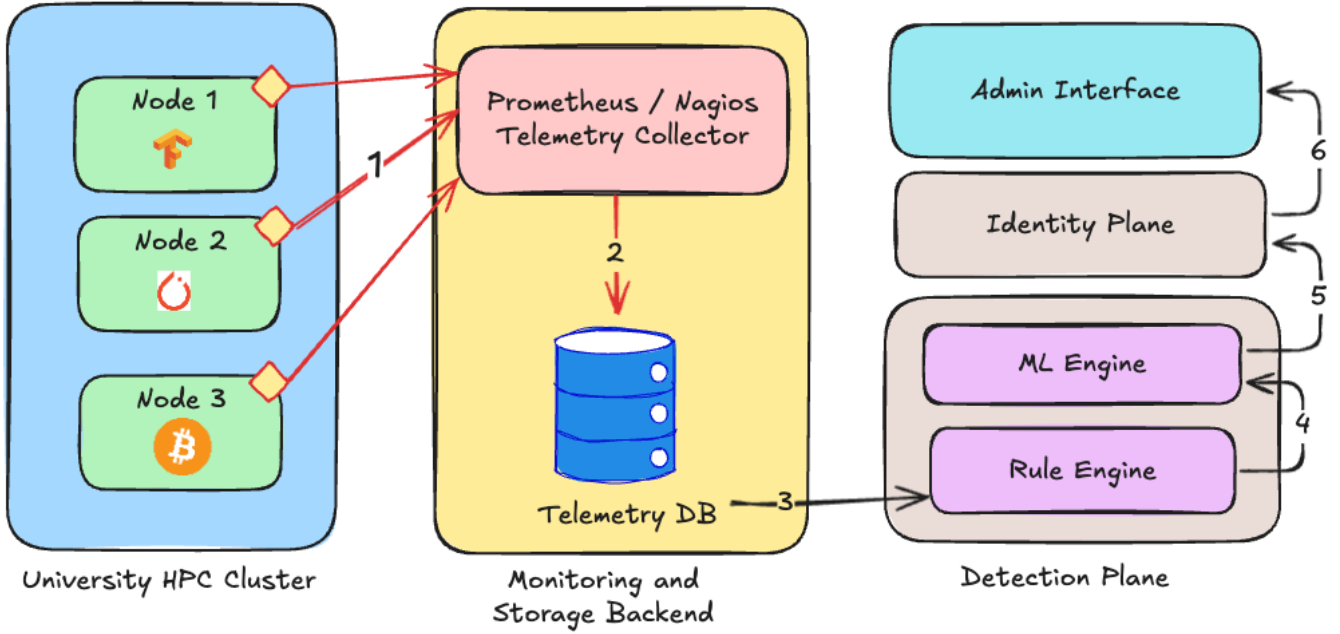


Figure 1: System architecture of the proposed cryptomining detection framework. The left column shows the University HPC Cluster with node-level agents collecting host telemetry. The center block illustrates the Monitoring and Storage Backend (Prometheus/Nagios and Telemetry DB). The right column represents the Detection Plane, including Rule and ML Engines, integrated with the Identity and Admin layers. Numbered links correspond to the data flows detailed in Table 1.

3 Proposed Solution

Our proposed solution adopts a hybrid framework that leverages both system metrics and network behavior to detect unauthorized cryptomining on university infrastructure. The key principle is that sustained compute activity alone does not imply mining; instead, mining workloads reveal themselves through persistent communication with remote mining pools during those compute-heavy intervals. The framework therefore unifies two complementary observations: (1) local resource consumption patterns at the host level and (2) outward network behavior that accompanies them.

4 System Architecture

Figure 1 illustrates the complete operational workflow of our proposed cryptomining detection system. The design emphasizes modularity, observability, and accountability by linking performance metrics, behavioral analytics, and user authentication within a unified framework. The workflow is divided into three core domains: the **University HPC Cluster**, the **Monitoring and Storage Backend**, and the **Detection Plane**.

The left column represents the **University HPC Cluster**, where students and researchers execute compute-intensive workloads such as deep learning model training or simulation experiments. Each HPC node runs a lightweight monitoring *agent* that periodically collects local telemetry — CPU/GPU utilization, memory footprint, I/O throughput, and network statistics — and forwards these metrics to the central monitoring service. This communication is denoted as **Link 1** in Figure 1.

At the center of the diagram is the **Monitoring and Storage Backend**, built atop industry-standard observability tools like Prometheus or Nagios. The telemetry collector aggregates incoming host metrics and stores them in a time-series **Telemetry Database** for downstream analytics (**Link 2**). The Telemetry DB serves as the authoritative data source for detecting temporal correlations between compute load and outbound network activity. The aggregated data is subsequently made available to the detection pipeline (**Link 3**).

On the right, the **Detection Plane** operates as a modular analytics pipeline. It contains two main analytical subsystems: a **Rule Engine** and an **ML Engine**. The Rule Engine performs lightweight heuristic checks to identify nodes exhibiting sustained high compute utilization and persistent external communication — a combination indicative of potential mining activity. Hosts that pass these initial rule filters are forwarded (**Link 4**) to the ML Engine, which applies learned temporal and statistical models to differentiate between benign high-performance research tasks and unauthorized mining.

Confirmed suspicious instances are passed to the **Identity Plane** (**Link 5**), where authentication and network logs (e.g., 802.1X, RADIUS, and NetID mappings) are queried to attribute the anomaly to a specific user or machine. Finally, validated alerts with contextual metadata — including confidence scores, time intervals, and host details — are sent to the **Admin Interface** (**Link 6**). Administrators can visualize detections, inspect historical trends, and initiate corrective or disciplinary measures.

By integrating host telemetry, behavioral detection, and user identity correlation, this architecture ensures precise and explainable identification of mining activity without disrupting legitimate research workloads. The modular separation between collection, analysis, and attribution layers further supports incremental deployment and scalability across departmental HPC sub-clusters.

5 Datasets

Accurate detection of cryptomining in university HPC environments requires datasets that capture both host and network telemetry under realistic workloads. Since mining activity can vary widely in network protocol, encryption, and host behavior, our framework leverages multiple datasets to build robust detection models. We employ one primary dataset for training the network-level component and supplement it with additional datasets for cross-validation and host-level calibration.

5.1 MineShark Cryptomining Traffic Dataset

Our primary dataset is the **MineShark Cryptomining Traffic Dataset** [14]. Released via Zenodo as part of an NDSS 2025 study, MineShark provides a large corpus of labeled network traffic representing various cryptomining algorithms (Bitcoin, Monero, Ethereum) and benign workloads. The dataset includes millions of flow records with metadata such as packet inter-arrival times, flow durations, payload sizes, and port distributions, making it suitable for both supervised and unsupervised detection. We employ this dataset for model training and validation of our network-level ML classifier, as it best reflects the temporal and statistical structure of real-world mining traffic.

5.2 Supplementary Datasets

To enhance generalization and ensure coverage across multiple communication patterns, we use two complementary datasets:

- **Encrypted Traffic Feature Dataset (ETFD)** [15]: a large dataset of encrypted benign and malicious flows that enables the model to learn general encrypted traffic distributions and improve robustness to obfuscation.
- **Cryptojacking Network Traffic 2021 (CNT21)** [16]: a small but well-labeled dataset distinguishing mining from benign traffic, used here primarily for initial feature validation and testing model portability across domains.

Each dataset is preprocessed to extract standardized flow-level features (duration, packet counts, inter-arrival time, entropy, and flow directionality) consistent with those proposed in Dong et al. [11] and Russo et al. [9]. We further normalize timestamps and align features to five-minute windows to synchronize with host-level telemetry.

5.3 Host-Level Dataset Augmentation

Because publicly available mining datasets rarely include host telemetry, we plan to augment existing data by capturing CPU, GPU, and I/O metrics from controlled experiments on the NYU HPC cluster. Following methodologies established in Gangwal et al. [7], we will simulate mining workloads under restricted conditions and label traces accordingly. This hybrid dataset will allow

joint training of host and network models, supporting our multi-layered detection framework.

Table 2 summarizes the datasets adopted for this work. The combination of large-scale network traces, smaller labeled cryptomining datasets, and in-house host-level captures enables comprehensive coverage of both behavioral and communication aspects of mining activity.

6 Implementation

This section describes the end-to-end implementation of the proposed cryptomining detection framework, focusing on how data is collected, processed, and analyzed across multiple system layers. The implementation is divided into four key analytical components: host-level analysis, network-level analysis, fusion-based detection, and evaluation.

6.1 Host-Level Analysis

The host-level component of our framework focuses on identifying computational patterns characteristic of cryptomining within the university’s HPC cluster. Each server runs a lightweight agent that continuously records system metrics to capture the node’s computational and process-level behavior. The challenge lies in distinguishing unauthorized mining—characterized by steady, repetitive, and compute-dominant activity—from legitimate high-performance workloads such as deep learning or simulations that exhibit bursty and heterogeneous resource usage.

6.1.1 Data Collection and Feature Extraction. Each HPC node executes a local monitoring agent that samples metrics from operating system and GPU interfaces (`/proc/stat`, `nvidia-smi`, and `perf`) every 15–60 seconds. These include CPU utilization, GPU occupancy, memory usage, disk I/O, and hardware performance counters (HPCs). Following Gangwal et al. [7], these counters are essential for identifying covert mining workloads through instruction-level patterns such as:

- **Integer vs. Floating-Point Instruction Ratio:** Mining workloads primarily use integer operations for hashing computations, while ML and simulation workloads exhibit a higher floating-point ratio.
- **Instructions Per Cycle (IPC) and Cache Misses:** Cryptominers show consistent IPC values and predictable cache behavior due to repetitive, non-branching loops.
- **Memory Bandwidth Utilization:** Mining tends to consume less memory bandwidth relative to compute intensity, distinguishing it from data-parallel training tasks.

These features have proven effective across multiple host-based detection studies, such as Carlin et al. [5], who applied dynamic instruction-trace profiling to differentiate mining binaries, and Caprolu et al. [10], who demonstrated that stable CPU power signatures correspond strongly with mining workloads.

6.1.2 Behavioral Differentiation. Mining workloads are further distinguished by their temporal uniformity. Whereas legitimate ML and simulation tasks generate bursty GPU activity interleaved with I/O and checkpointing, cryptominers sustain near-constant utilization. Following the approach of Russo et al. [9] and Veselý et al. [12], our framework proposes a sliding window (e.g., 10 minutes)

Table 1: Labeled data flows in Figure 1.

No.	Description
1	Agents on each HPC node send host telemetry to the Prometheus or Nagios telemetry collector. Typical contents include CPU and GPU utilization, memory and I/O statistics, and process metadata at fixed intervals.
2	The monitoring service writes normalized and time aligned metrics to the telemetry database for downstream analytics and historical queries.
3	The detection pipeline reads aggregated metrics and flow summaries from the telemetry database as its primary input stream.
4	The rule engine forwards candidate hosts and time windows to the ML engine after simple checks for sustained compute together with long lived low throughput connections.
5	The ML engine sends confirmed events to the identity plane to resolve the responsible user and machine using 802.1X or RADIUS authentication logs, DHCP leases, and NetID login records.
6	The identity plane publishes attributed alerts to the administrator interface or SIEM, including evidence bundles such as utilization timelines, top flows, and confidence scores.

Table 2: Summary of datasets used for training and evaluation.

Dataset	Source	Content Type	Use in Framework
MineShark	Zenodo	Labeled network flows (mining + benign)	Primary training and evaluation dataset for network-level ML detection.
Encrypted Traffic Feature Dataset (ETFD)	Mendeley Data	Encrypted benign/malicious flows	Pretraining and robustness testing against encrypted mining.
Cryptojacking Network Traffic 2021 (CNT21)	Mendeley Data	Small labeled mining and benign flows	Feature validation and cross-domain testing.
NYU HPC Host-Mining Dataset (proposed)	In-house collection	Host metrics: CPU/GPU, process behavior, system counters	Joint host-network model training and correlation testing in campus environment.

for each node and computes the coefficient of variation (CV) of GPU utilization. Nodes with CV below a threshold (e.g., < 0.05) across multiple windows are flagged for potential mining. Complementary indicators include:

- Persistent execution of the same binary or container image over long intervals.
- High CPU/GPU load with minimal disk or memory I/O.
- Low process entropy, reflecting few active binaries or threads.

Such patterns mirror the host-level findings reported by Gangwal et al. [7] and Carlin et al. [5], which show that mining workloads maintain steady-state CPU and GPU utilization with negligible variance.

6.1.3 Statistical and Anomaly Modeling. To identify deviations from legitimate research activity, our system constructs an unsupervised baseline for each node using exponentially weighted moving averages (EWMA) of historical performance metrics. Deviations from this baseline are assigned anomaly scores. These anomaly scores, combined with rule-based detections, are passed to the detection plane for further validation. This approach parallels the host-level anomaly detection pipeline proposed in Darabian et al. [6], where dynamic and static feature fusion achieved high accuracy in classifying mining malware.

6.1.4 Integration with Monitoring Backend. All telemetry streams are transmitted to the Prometheus/Nagios collector for centralized processing, consistent with methodologies proposed by Shi et al. [13]. Each sample is labeled with node and timestamp metadata, allowing correlation with authentication logs in subsequent layers. By combining hardware performance counter metrics [7], behavioral stability indicators [5, 9], and anomaly-driven thresholds [6], our host-level analysis provides an interpretable yet effective foundation for the broader cryptomining detection framework deployed in the university’s HPC environment.

6.2 Fusion and Detection Pipeline

The final analytical layer integrates the host- and network-level features into a unified detection pipeline composed of two complementary subsystems: a **Rule Engine** for deterministic screening and a **Machine Learning (ML) Engine** for probabilistic classification. The fusion process ensures that suspicious events are evaluated in the context of both compute behavior and communication activity, thereby reducing false positives and providing higher interpretability.

6.2.1 Feature Fusion and Temporal Alignment. Telemetry from the host and network layers is aligned on a fixed temporal window—typically five minutes—based on timestamps from the Prometheus database. For each node n , the fused feature vector $\mathbf{X}_n$ is represented

as:

$$\mathbf{X}_n = [\mathbf{h}_n, \mathbf{f}_n]$$

where $\mathbf{h}_n$ is the host-level feature vector (CPU/GPU utilization, I/O metrics, process entropy, HPC ratios) and $\mathbf{f}_n$ is the aggregated network feature vector (flow duration, packet inter-arrival variance, destination entropy, byte ratio). This design mirrors the hybrid approaches used in Veselý et al. [12] and Shi et al. [13], who demonstrated that combining host and network attributes significantly improves detection confidence under encryption and obfuscation.

6.2.2 Rule-Based Detection Layer. The Rule Engine executes deterministic checks based on empirically defined thresholds drawn from both telemetry sources. A node is flagged as suspicious if it satisfies at least one composite condition, such as:

- *Sustained compute load:* GPU utilization > 90% with coefficient of variation (CV) < 0.05 for three or more windows.
- *Low I/O and high compute ratio:* Compute-to-I/O ratio > 10^2 , consistent with hash-based mining tasks.
- *Persistent outbound connections:* One or more TCP flows exceeding one hour of duration or recurrent reconnections to a fixed IP pool.

These rules act as a high-recall filter, rapidly identifying potential mining cases while minimizing computational overhead. Nodes that pass the rule-based screening are forwarded to the ML Engine for probabilistic classification.

6.2.3 Machine Learning Engine. The ML Engine leverages supervised models trained on the MineShark [14] and supplementary datasets described in Section 2. The feature set includes statistical summaries (mean, variance, entropy), temporal stability measures, and HPC-based ratios. We primarily employ Random Forest (RF) and Gradient Boosted Decision Trees (GBDT), as they perform well on heterogeneous tabular data. The classifier outputs a mining probability score $P_{\text{mine}}(n)$ for each node, which is thresholded to balance precision and recall.

The training follows a stratified 80/20 split for validation, using metrics such as accuracy, F1-score, and false-positive rate. To address dataset imbalance, we apply SMOTE oversampling and cost-sensitive weighting. The ML Engine is periodically retrained using feedback from administrators, consistent with the adaptive learning approach proposed by Caprolu et al. [10].

6.2.4 Confidence Scoring and Escalation. For each alert, a combined confidence score is computed as a weighted ensemble of rule-based and ML outputs:

$$C(n) = \alpha R(n) + (1 - \alpha) P_{\text{mine}}(n)$$

where $R(n) \in \{0, 1\}$ is the binary result of rule evaluation and α is the confidence weight (default $\alpha = 0.3$). Nodes with $C(n) > \tau$ are escalated to the identity plane for user attribution. This fusion approach balances interpretability (through deterministic rules) and generalization (through learned patterns), consistent with prior hybrid systems by Darabian et al. [6] and Gangwal et al. [7].

6.2.5 Adaptive Feedback and Continuous Learning. Each confirmed alert from the administrator dashboard is logged back into the Telemetry DB with its true label, enabling incremental retraining of the model. This self-updating loop ensures continuous adaptation

to evolving mining strategies and new benign workloads. Periodic evaluations against updated datasets ensure sustained accuracy and low false-positive rates across HPC workloads.

Overall, this fusion and detection pipeline forms the decision core of our framework, unifying statistical heuristics and machine intelligence into an interpretable and scalable cryptomining detection system.

7 Evaluation and Validation

To verify the effectiveness and robustness of our proposed hybrid cryptomining detection system, we plan a multi-stage evaluation framework combining quantitative performance metrics, qualitative inspection, and longitudinal deployment in the NYU campus network. This section outlines the objectives, metrics, and experimental methodology used to validate our approach.

7.1 Evaluation Objectives

We aim to evaluate our framework along three primary dimensions:

- **Detection Accuracy:** Measure the ability of the system to correctly identify cryptomining workloads across heterogeneous university computing environments, including GPU-accelerated nodes, CPU-based servers, and general-purpose research machines.
- **False Positive Minimization:** Ensure legitimate compute-intensive activities such as ML training and simulation workloads are not incorrectly flagged, thus maintaining operational transparency for students and researchers.
- **Scalability and Real-Time Responsiveness:** Evaluate how efficiently the system scales across hundreds of monitored nodes and whether its latency remains acceptable for near real-time threat detection.

7.2 Experimental Design

We will validate the framework using both controlled and real-world scenarios. Controlled experiments will simulate cryptomining workloads on isolated HPC nodes using mining software such as *XMRig* and *PhoenixMiner* under different network and hardware configurations. Corresponding benign workloads will include TensorFlow-based model training, data-intensive simulations, and large-scale file transfers. Each experiment will be monitored through the same telemetry agents deployed in the production system.

Network traces will be drawn primarily from the MineShark [14] dataset and augmented with live packet captures from NYU’s research servers. Host-level telemetry will be logged through Prometheus and GPU drivers (NVIDIA SMI), capturing CPU/GPU utilization, memory, and I/O statistics. All data will be time-synchronized to support cross-layer correlation analysis.

7.3 Performance Metrics

To ensure objective validation, we will compute standard detection metrics used in the literature [7, 9, 10]:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$\text{F1-score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

where TP , TN , FP , and FN represent true/false positives and negatives respectively. In addition, we will evaluate:

- **ROC-AUC:** to quantify trade-offs between detection sensitivity and specificity.
- **Mean Detection Latency:** the time between mining onset and alert generation.
- **Resource Overhead:** average CPU and memory consumption of the monitoring agent to ensure scalability across student nodes.

7.4 Cross-Validation and Ablation Studies

The model will be trained using stratified k -fold cross-validation ($k = 5$) to assess generalizability. Ablation studies will isolate the impact of each feature subset—host-only, network-only, and fused—to quantify the marginal contribution of each data modality. We expect that the hybrid model will outperform single-modality baselines, consistent with results reported in Veselý et al. [12] and Shi et al. [13].

7.5 Deployment Validation

Following offline validation, the system will undergo a limited pilot deployment across a subset of NYU’s HPC cluster nodes. Live alerts will be correlated with user identities through the 802.1x-based authentication system described earlier. Feedback from administrators will be incorporated through a continuous learning loop, retraining models based on confirmed alerts and benign workload logs. The long-term objective is to achieve over 95% accuracy with less than 1% false positive rate in real-world operation.

7.6 Expected Outcomes

We expect the hybrid approach to provide both interpretability and generalization. The rule-based layer should capture explicit violations (e.g., sustained high utilization with minimal I/O), while the ML classifier refines detection for ambiguous patterns such as encrypted or throttled mining. Overall, the system aims to deliver a scalable and explainable framework capable of maintaining fairness between security enforcement and academic freedom.

8 Conclusion

In this work, we presented a hybrid, multi-layered framework for detecting illicit cryptomining activities within university computing infrastructures. The proposed system combines host-level telemetry, network traffic analysis, and user authentication metadata to identify abnormal resource utilization indicative of mining workloads.

Our approach bridges the interpretability of rule-based heuristics with the adaptability of machine learning, enabling precise differentiation between legitimate compute-intensive tasks and unauthorized mining behavior.

We formulated the detection pipeline to operate on real-time data collected from existing monitoring tools such as Prometheus and Nagios, integrating both statistical metrics—such as sustained GPU load, compute-to-I/O ratio, and process persistence—and learned features from network flow patterns. The result is a scalable system capable of analyzing large volumes of traffic and telemetry without introducing significant operational overhead.

Through our planned evaluation—combining controlled experiments, cross-validation with large-scale datasets like MineShark, and in-situ deployment on the NYU HPC cluster—we aim to demonstrate that this hybrid framework can achieve high accuracy (targeting over 95%) while maintaining a false positive rate below 1%. More importantly, our system is designed with transparency and privacy in mind, ensuring that the detection process remains accountable and auditable within an academic context.

Future work will focus on extending this system to support on-line adaptive retraining, integration with campus-wide security dashboards, and broader applicability to cloud or edge computing environments. By fusing deterministic rules with data-driven intelligence, our framework establishes a generalizable foundation for responsible resource governance in research and educational institutions.

References

- [1] Galaxy Digital Holdings LTD. 2024 bitcoin mining mid-year report: The rise of the gigawatt. https://assets.ctfassets.net/h62aj7eo1csj/6aeeTUuknj37LXZmfZzTRb/a69b7635cac3880ef594bdc07151eec0/GLXY_2024_Whitepaper_BTC-Mining-Mid-Year_v03.pdf, July 2024. Accessed 2025-10-28.
- [2] Coin Metrics. State of the network: Data on bitcoin mining revenue and network fundamentals. <https://coinmetrics.io/state-of-the-network/>, 2024. Accessed 2025-10-28. Primary dataset referenced by BuyBitcoinWorldwide’s 2024 mining statistics.
- [3] The Economic Times. Bitcoin mining costs \$1,320 in iran but a whopping \$306,000 in italy. <https://economictimes.com/news/international/us/bitcoin-mining-costs-1300-in-iran-but-a-whopping-306000-in-italy-heres-the-full-list/articleshow/123656866.cms>, 2024. Accessed 2025-10-28.
- [4] Digiconomist. Bitcoin energy consumption index. <https://digiconomist.net/bitcoin-energy-consumption/>, 2024. Accessed 2025-10-28.
- [5] Dohnall Carlin, Paul O’Kane, Sakir Sezer, and John Burgess. Detecting cryptomining using dynamic analysis. In *Proceedings of the 2018 International Conference on Privacy, Security and Trust (PST)*, pages 1–6, 2018.
- [6] Hadi Darabian, Sajad Homayounoot, Ali Dehghantanha, Sattar Hashemi, Hamed Karimipour, Reza M. Parizi, and Kim-Kwang Raymond Choo. Detecting cryptomining malware: A deep learning approach for static and dynamic analysis. *Journal of Grid Computing*, 18(2):293–303, 2020.
- [7] Ankit Gangwal, Samuele Giuliano Piazzetta, Gianluca Lain, and Mauro Conti. Detecting covert cryptomining using hardware performance counters. In *Proceedings of the 19th International Conference on Cryptology and Network Security (CANS 2020)*, volume 12579 of *Lecture Notes in Computer Science*, pages 317–336. Springer, 2020.
- [8] Rupesh Raj Karn, Prabhakar Kudva, Hai Huang, Sahil Suneja, and Ibrahim M. Elfadel. Cryptomining detection in container clouds using system calls and explainable machine learning. *IEEE Transactions on Parallel and Distributed Systems*, 32(3):674–691, 2021.
- [9] Michele Russo, Nedim Šrđić, and Pavel Laskov. Detection of illicit cryptomining using network metadata. *EURASIP Journal on Information Security*, 2021(1):11, 2021.
- [10] Maurantonio Caprolu, Simone Raponi, Gabriele Oligieri, and Roberto Di Pietro. Cryptomining makes noise: Detecting cryptojacking via machine learning. *Computer Communications*, 171:126–139, 2021.
- [11] Lijian Dong, Zhigang Li, Xiangrong Li, Xiaofeng Wang, and Yuan Liu. Identification technique of cryptomining behavior based on traffic features. *Frontiers in Physics*, 11:1269889, 2023.

- [12] Vladimír Veselý and Martin Žádník. How to detect cryptocurrency miners? by traffic forensics! volume 31, pages 1–14, 2019.
- [13] Haoran Sun, Ruisheng Shi, Lina Lan, Zhiyuan Peng, and Chenfeng Wang. A two-stage encrypted cryptomining traffic detection mechanism in campus network. pages 602–610, 2024.
- [14] MineShark Project Authors. Mineshark: Cryptomining traffic detection at scale. Zenodo, 2025. Includes large-scale labeled network traffic for mining and benign workloads.
- [15] K. Meidan, O. Kfir, A. Shabtai, and Y. Elovici. Encrypted traffic feature dataset (etfd). Mendeley Data, v1, 2022. Dataset of encrypted benign and malicious traffic for network-based ML analysis.
- [16] Maurantonio Caprolu, Simone Raponi, Gabriele Oligeri, and Roberto Di Pietro. Cryptojacking network traffic 2021 (cnt21). Mendeley Data, v1, 2022. Contains labeled cryptomining and benign traffic flows with packet-level features.